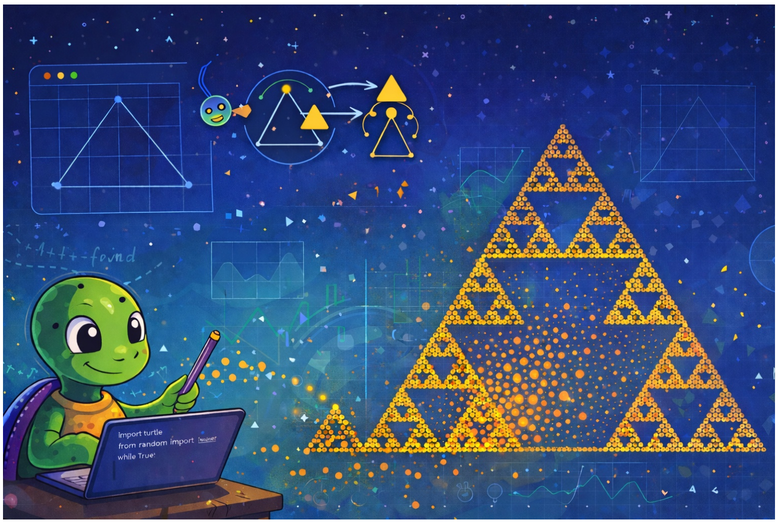
# Sierpiński



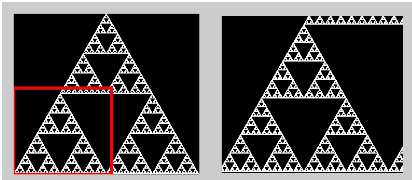

This is no starter for this project
You can start with a completely empty Python file.

This works a locally installed editor, such as Thonny or the Raspberry Pi online editor:

- editor.raspberrypi.org

## 1

A **fractal** is a pattern that keeps repeating as you try to zoom in and never ends.



The **Sierpiński triangle,** or **Sierpiński gasket,** is a triangular fractal pattern. This is shown in the picture. If you zoom into the red box in the left picture you get the right picture, and you can do this forever.
We are going to write a program to make the Sierpiński triangle and it will seem as though we are doing it randomly!

## 2

Open a new Python program and add the lines on the right.

- The `import turtle` command says we are going to use all of the turtle graphics commands.
- The `from random import randrange` says we are only going to use the randrange command from the random library.

```
import turtle
from random import choice
```

## 3

First of all,

- Choose a point on the screen. Later this will be one corner of the triangle.
- Then turn the turtle towards that point and move half-way towards it.

```
import turtle
from random import choice

point = (100, 100)
angle = turtle.towards(point)
print("The angle to the point is", angle, "degrees")

print("Turning the turtle towards that point ...")
turtle.setheading(angle)

print("Drawing a line in that direction ...")
turtle.forward(100)
```

**Note:** The `print` commands are just there to show you what is happening. They will be removed in later code examples, and you can remove them too.

## 4

We don't want to draw lines, and to draw the Sierpiński triangle we will want to move forward half the distance to the point, not 100 every time.

- Use `penup` to stop the line being drawn.
- Find half the distance from the turtle to the point, and move forward that amount.
- Draw a dot when the the turtle arrives.
- Do this 10 times and see the dotted pattern you get.

```
import turtle
from random import choice

turtle.penup()

point = (100, 100)
for i in range(10):
    angle = turtle.towards(point)
    turtle.setheading(angle)

    distance = turtle.distance(point) / 2
    print("Half the distance to the point is", distance)

    turtle.forward(distance)
    print("Drawing a new dot ...")
    turtle.dot(5)
```

Notice that these two lines

```
    angle = turtle.towards(point)
    turtle.setheading(angle)
```

are not new lines but they are now indented with spaces at the start. This is important to make sure they are in the `for` loop.

## 5

The final step is to have three corners, and for each dot we will pick a corner to travel toward at random.

- Create an array of three corner points.
- Each time round the for loop pick a random corner with the `choice` command. This comes from the `random` library.
- Make the turtle super fast by setting the speed to 11.
- Draw 1000 dots instead of 10.

```
import turtle
from random import choice

turtle.penup()
turtle.speed(11)

corners = [
        (-200, -200),
        (0, 100),
        (200, -200)
    ]

for i in range(1000):
    point = choice(corners)

    angle = turtle.towards(point)
    turtle.setheading(angle)

    distance = turtle.distance(point) / 2

    turtle.forward(distance)
    turtle.dot(5)
```

## 6

Some things to try:

- 1000 dots may still not be enough to show the pattern well. Try more, or even telling it to go on forever.
- The dots can be a bit blobby. Can you work out how to make the dots smaller so that the pattern is clearer?
- Try different corners to make different triangles.
- **(Tricky)** What happens if you have a different colour for when you go towards each corner? This is tricky because you will need to make changes to the corners array or how you pick the corners. There are different ways to do this!